Neural Networks

Model

- neuron takes vector of inputs $x = [x_1, x_2, ..., x_n]^T$ and produces single output y
- every input x_i is multiplied by it's corresponding weight w_i, which gives **net activation**:

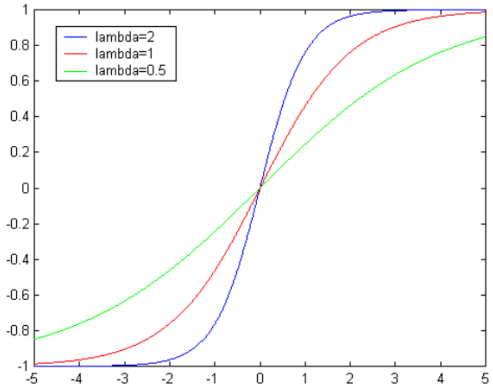
$$\text{net} = w^T x = \sum_{i=1}^{n} w_i x_i$$

- output is calculated by passing net activation through **activation function** f:

$$y = f(\text{net}) = f(w^T x)$$

  - two output convensions - **bipolar** $y \in [-1, 1]$ and **unipolar** $y \in [0, 1]$
  - two main function types
    - **discrete**: $f(\text{net}) = \text{sgn}(\text{net})$ - hard yes/no decision
    - **continiuous (sigmoid)** - with λ parameter:

$$f(\text{net}) = \frac{2}{1 + \exp(-\lambda \text{ net})} - 1$$



- $x_n = 1$ - **bias trick**, that enables us to treat w_n as bias for neuron, which simplifies notation

Learning

- **batch learning** - weights updated after one training epoch
- **incremental learning** - weights updated after each sample
- **supervised/unsupervised learning** - with or without target signal

Perceptron rule

- for supervised learning of **discrete** neurons:

$$r = d_i - y_i = d_i - \text{sgn}(w_i^t x)$$

$$\Delta w_i = c[d_i - \text{sgn}(w_i^T x)]x$$

  - if output is correct ($y_i = d_i$) then $r = 0 \rightarrow$ no change to weight
  - else ($r = \pm 2$) then weight is changed to correct error by $\pm 2cx_j$

Delta rule

- for supervised learning of **continiuous** neurons:

$$r = \delta = [d_i - f(w_i^T x)] f'(w_i^T x)$$

$$\Delta w_i = c[d_i - f(w_i^T x)] f'(w_i^T x) x$$

- using **derivative of the activation function** to update weight according to error gradient

One layer network

- J inputs $y_1, ..., y_J$ feed into K output neurons ($J \times K$ weights), that are producting outputs of $z_1, ..., z_K$
- **error function** - for a single sample y_l with target $d = [d_1, ..., d_K]^T$

$$E_l = \frac{1}{2} \sum_{k=1}^{K} (d_{lk} - z_{lk})^2$$

- we modify weights in the opposite direction to error gradient: $w^{k+1} = w^k - \eta \Delta E(w^k)$
  - η - learning rate parameter
- for bipolar sigmoidal function there is an elegant identity:

$$f'(\text{net}_k) = \frac{1}{2}(1 - z_k^2)$$

  - derivative calculated **only** from output
- sigmoidal function has a **vanishing gradient** problem - even if error $d - z$ is large, derivative at this point is near 0

Learning algorithm

- simple algorithm:
  1. Choose η (learning rate) and E_{max} (maximum acceptable error value)
  2. Initialize weights W
  3. Reset cumulative error $E_{\text{sum}} = 0$
  4. For each training sample:
     1. Compute the network output
     2. Add the sample's error to E_{sum}
     3. Update weights
  5. IF $E_{\text{sum}} < E_{\text{max}}$ stop, otherwise go to 3
- stopping criterion in simple algorithm is prone to overfitting, it's better to **stop early on a validation set**:
  5. Save previous validation error $E_{\text{val_ref}} = E_{\text{val}}$
  6. Compute classification error on validation set E_{val}
  7. If $E_{\text{val}} > E_{\text{val_ref}}$ stop, otherwise go to 3

Multi layer network

- able to represent **nonlinear decision boundries**
- I input neturons feed to J "hidden" neurons, which feed to K output neurons
- problem - hidden neurons don't have target values
- sollution - **backpropagation**
  - error signal signal of hidden neuron is a weighted sum of the deltas of the neurons it feeds into, multipled by it's own derivative
  - we calculate error signals at the last layer before output from target values, and then propagate it back:

$$\Delta v_{ij} = \eta \left(\sum_{k=1}^{K} \delta_{zk} w_{k_j} \right) f_{j'}(\text{net}_j) x_i$$

Learning algorithm

1. Choose η (learning rate) and $E_{\max}$ (maximum acceptable error value)
2. Initialize W and V (weights between all layers)
3. Reset E_{sum}
4. For each training sample:
    1. Compute the networks output
    2. Add the sample's error to E_{sum}
    3. Compute error signal vectors
    4. Update weights to all layers
5. Check stopping (again, preferably on validation set)

Learning parameters tips

- update timing
    - incremental - after each sample
    - batch - update once per epoch, you sum Δw across all samples and then perform update
- escaping local minimas
    - adding random perturbations to the weights
    - adding noise to inputs
    - randomizing order of training samples each epoch
- initial weights
    - trying multiple random initializations and keeping the best
    - if weights are too large, neurons saturate, $f' = 0$, so learning stops
    - if weights are all 0, output is undefined
    - general rule - set random weights so that total net activation is about 1
- learning rate
    - too small - slow learning, trapping in local minimas
    - too large - oscilations
    - solution - variable learning rate, increasing by constant value if error systematically decreases, decreasing geometrically when error increases

Problems with classic neural networks

- classic neural network - **fully-connected (FC) architecture**
- **vanishing gradients** - error signal vanishes geometrically while backpropagating, resulting in little to none training for the first layer
- **spatial structure** - when for example computing images, connecting all neurons to every pixel value is unnatural and wasteful (pixels in the top-right rarely are correlated with those in the botton-left)
- idea - first layers should work as feature extractors looking at small areas

Convolution

- in convolutional neural networks (**CNN**) filter has smaller size than input
- 3 key characteristics:
    - locality of connections - each output neuron is connected only to a samll patch of input (**receptive field**)
    - parameter sharing - same filter (same set of weights) is slid across whole picture

    - translation equivariance - same filter works the same in the whole output (cat detector in the top-right corner works also as a cat detector in the bottom-left)
- mathematical definition:
    - you have kernel K (for example 2×2 weights w, x, y, z) on top of the image at position (i, j)
    - you multiply input values by coresponding kernel values, then sum them
    - then you slide the kernel and repeat
    - $$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(i+m, j+n) K(m,n)$$
- parameters:
    - f - *filter size* (e.g 2×2, $4 \times 2 \times 3$)
    - s - *stride* - step length, how big of a step filter does
    - p - *padding* - extra border around the input (usually $p = \frac{f}{2}$)

Padding

- output size is:

$$\text{size} = \frac{n - f + 2p}{s} + 1$$

- two regimes:
    - *valid padding* ($p = 0$) - output of the convolution is smaller than input
    - *same padding* ($p = \frac{f}{2}$) - output is the same size as input
- strategies:
    - *zero padding* - surrounding image with zeroes - simplest but adds artificial black border feature (but still most popular in image networks)
    - *replicated padding* - copy the neares value outward - avoids artificial zeroes
    - *symmetric padding* - mirrors the image at the boundry - best preserving local features
    - *circulated padding* - copy-and-paste image next to the input - usefull when data is periodic

Linearity

- convolution is a linear operation
- stacking linear operations gives another linear operation - we need to introduce non-linearity, or network collapses to single linear classifier
- CNN's use simpler functions than sigmoid:
    - **ReLU**

$$f(x) = \max(0, 1)$$

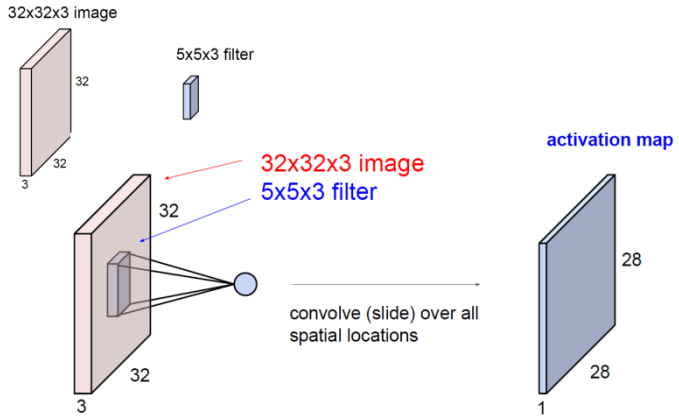
        - for $x > 0$ gradient is exactly 1, so backpropagated signal does not vanishes
        - trivial to compute
        - **dying ReLU problem** - if neuron consistently outputs negative value, it's gradient is zero forever and it stops learning
    - **Leaky ReLU**

$$f(x) = \max(\alpha x, x)$$

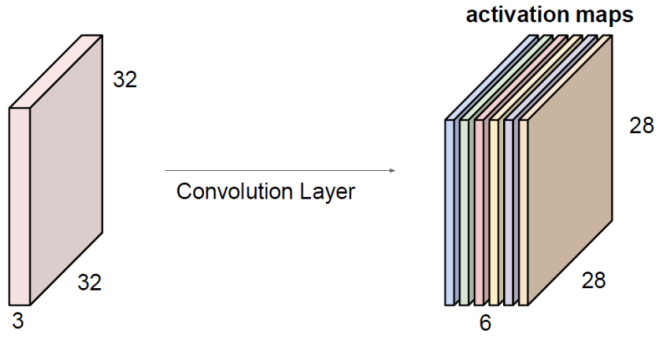
        - even if neuron is off, small gradient flows through it

Building layers

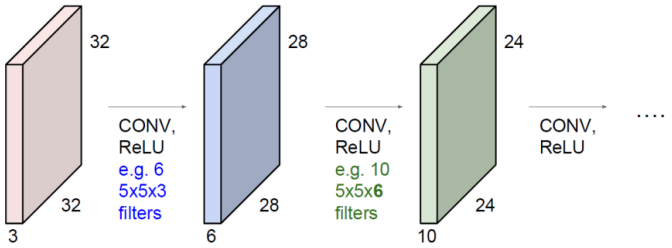
- each filter that processed whole image gives **activation map**:



  - $32 \times 32 \times 3$ image
  - $5 \times 5 \times 3$ filter
  - $28 \times 28 \times 1$ activation map
- when we add another, independent filters, it gives another $28 \times 28 \times 1$ activation map, that stacks behind first one
- for example for six filters we get $28 \times 28 \times 6$ convolutional layer:



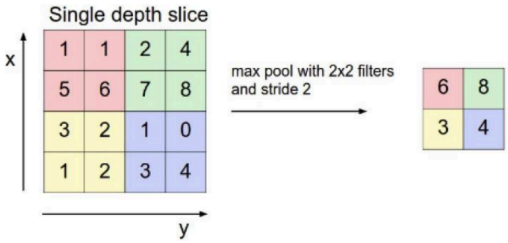
- then we can stack convolutional layers, by applying filters on convolutional layers, receiving another convolutional filter;



  - without padding, dimenshions shrink
  - filter depth grows (it must span full depth of previous layer's output)
- filter of size $1 \times 1 \times$ depth calculates dot product at each position - usefull to decrease depth without changing spatial size

Pooling

- **pooling** - reducing width and height while keeping depth unchanged
  - it does **not** mix channels - it operates on each depth slice independently
- **max pooling** - keeping only the maximum value in each pooling window:



- why pool at all:
  - improve computational efficiency (less to compute)
  - small shifts in input does not change the pooled output much

Normalization

- two main approaches:
  - **local response normalization (LRN)** - inhibiting neighbours of the neuron with high output value (abandoned approach)
  - **batch normalization (BN)** - for every mini-batch, batch mean is subtracted and divided by batch standard deviation, channel by channel, then it's scaled and shifted
    - stabilizes the distribution of activations during training
    - allows much higher learning rate
    - often makes training 5-10 times faster
    - practically every modern CNN uses it

Regularization

- **L1 regularization** - defining penalty as $\lambda \ |w|$ - encourages sparse weights, effectively selects a subset of inputs and ignores noisy operations
- **L2 regularization** - defining penalty as $\frac{\lambda}{2}w^2$ - penalizes large weights
- **Elastic net** - combination between L1 and L2 - $\lambda \ |w| + \lambda w^2$
- **Dropout**
  - different type of regularization
  - randomly sets outputs of fraction (typically 50%) of neurons to zero
  - each mini-batch of data trains a different, thinned sub-network
  - neurons are less relying on each other, so each must learn an independently usefull feature
  - adding noise

Transfer learning

- reusing what one network learned to help with a smaller one
- recipe:
  1. Take a pre-trained network
  2. Remove last classification layer (or last few layers)
  3. Add a new classifier for your task
  4. Either:
     - freeze the pre-trained weights, train only the new head
     - continue training the whole network at a low learning rate
- three questions:
  - **what** to transfer - low level features transfer easily, task specific does not
  - **how** to transfer - feature extraction, fine-tuning, domain adaptation
  - **when** to transfer - only when source and target tasks are related (**negative transfer** - missmatch between source and task, resulting in worse network than without transfer learning)